

Registro de conversación con IA

Proyecto: chatbot SPA responsive para chatear con un personaje ficticio usando Gemini (HTML, CSS y JavaScript)

Participantes: Kevin (estudiante) y Claude (asistente de IA de Anthropic)

Fecha de exportación: 1 de octubre de 2026

Transcripción completa, sin recortes, en el orden original de la conversación.

Kevin — mensaje 1

Hola, estoy haciendo un proyecto de un Chat bot para chatear con un personaje ficticio usando la IA, El proyecto tiene que ser una Single Page Aplicación responsive, cuando te pregunte respecto a una función de la aplicación quiero que me expliques paso a paso la función y la aplicación de esta en mi código, ten en cuenta que soy programador nivel principiante para darme las respuestas, aún así dame las opciones más eficientes o funcionales según el caso

Claude — respuesta

¡Hola, Kevin! Suena como un proyecto muy entretenido. Entendido cómo vamos a trabajar: cuando me preguntes por una función, te la explico paso a paso, en lenguaje de principiante, y te muestro cómo encaja en tu código. Siempre te propondré la opción más eficiente o práctica para tu caso. Si hay varias alternativas razonables, te diré cuál recomiendo y por qué.

Para que las explicaciones se ajusten a tu proyecto, me sirve saber con qué lo estás construyendo. Por ejemplo: HTML, CSS y JavaScript puro, o un framework como React. También si ya tienes elegida la API de IA que vas a usar (Claude, OpenAI, Gemini, etc.) y si tienes un backend o todo corre en el navegador.

Cuando quieras, pregúntame por la primera función o pega el código que ya tengas.

Kevin — mensaje 2

Solo HTML, CSS y JS, sin frameworks y voy a usar gemini ya que el proyecto lo requiere

Claude — respuesta

Perfecto, HTML, CSS y JavaScript puro con la API de Gemini es una combinación sencilla y suficiente para una SPA de chat.

Antes de empezar, hay un punto importante. Si llamas a Gemini directamente desde el JavaScript del navegador, tu API key queda visible para cualquiera que abra las herramientas de desarrollador de la página. Para un proyecto académico que corre en tu computador o que solo muestras en clase, eso suele ser aceptable. Si vas a publicar la página en internet, lo recomendable es poner un pequeño intermediario (por ejemplo, una función serverless) que guarde la clave y haga la llamada por ti. Cuando llegues a esa parte te explico cómo, si lo necesitas.

Una estructura base que te va a funcionar bien:

```
proyecto/  
├── index.html  → estructura: área de mensajes, campo de texto, botón enviar  
├── style.css   → diseño responsive  
└── app.js     → lógica: enviar mensaje, llamar a Gemini, mostrar respuesta
```

Cuando quieras, pregúntame por la primera función. Por ejemplo, cómo enviar un mensaje a Gemini, cómo darle la personalidad al personaje o cómo mostrar los mensajes en pantalla.

Kevin — mensaje 3

Como hago para obtener solo el texto limpio para imprimirlo en el chat?

Claude — respuesta

Cuando Gemini te responde, no te devuelve solo el texto: te devuelve un objeto JSON con varias capas. El texto que quieres mostrar está dentro de esas capas, así que el trabajo es "entrar" hasta él

y luego mostrarlo bien en pantalla.

Paso 1: entender la forma de la respuesta

La respuesta de Gemini se ve más o menos así (simplificada):

```
{
  "candidates": [
    {
      "content": {
        "parts": [
          { "text": "¡Hola, viajero! ¿Qué te trae por aquí?" }
        ]
      }
    ]
  }
}
```

La ruta hasta el texto es: candidates → el primero [0] → content → parts → el primero [0] → text.

Paso 2: extraer el texto de forma segura

Esta función saca el texto y, si algo viene vacío o falla, devuelve un mensaje en vez de romper tu aplicación:

```
function obtenerTexto(data) {
  const partes = data?.candidates?.[0]?.content?.parts;
  if (!partes) return "El personaje no pudo responder. Intenta de nuevo.";
  return partes.map(p => p.text || "").join("");
}
```

El ?. significa "si esto existe, sigue; si no, detente sin error". Unir todas las parts con join es más seguro que tomar solo la primera, porque a veces la respuesta llega dividida en varias partes.

Paso 3: quitar el formato Markdown

Gemini suele responder con asteriscos y símbolos como **negrita** o # títulos. La opción más eficiente es pedirle desde el principio que no los use, en la instrucción del personaje que envías en la petición:

```
systemInstruction: {
  parts: [{ text: "Eres [tu personaje]. Responde siempre en texto plano, sin Markdown, sin asteriscos ni símbolos de formato." }]
}
```

Como respaldo, por si igual se cuela algún símbolo, puedes limpiarlo así:

```
function limpiarMarkdown(texto) {
  return texto
    .replace(/\*\*(.*?)\*/g, "$1") // **negrita**
    .replace(/\*(.*?)\*/g, "$1") // *cursiva*
    .replace(/^#\s*/gm, "") // # títulos
    .trim();
}
```

Paso 4: imprimirlo en el chat

Usa textContent en lugar de innerHTML. Así, si la respuesta trae algo parecido a código HTML, se muestra como texto y no se ejecuta, lo que protege tu página:

```
function mostrarMensaje(texto) {
  const burbuja = document.createElement("div");
  burbuja.className = "mensaje bot";
  burbuja.textContent = texto;
  document.getElementById("mensajes").appendChild(burbuja);
}
```

Y en tu CSS agrega esto para que se respeten los saltos de línea de la respuesta:

```
.mensaje { white-space: pre-wrap; }
```

Cómo se conecta todo en tu código

Después de hacer la petición a Gemini, lo usarías así:

```
const data = await respuesta.json();
const texto = limpiarMarkdown(obtenerTexto(data));
mostrarMensaje(texto);
```

Si me muestras cómo tienes hecha la petición a Gemini, te indico exactamente dónde colocar cada parte.

Kevin — mensaje 4

Aún no lo conecto a gemini, como hago que antes de pulsar enviar si el texto está vacío no se envíe nada?

Claude — respuesta

La idea es revisar el contenido del campo de texto antes de hacer cualquier cosa. Si está vacío (o solo tiene espacios), la función se detiene y no envía nada.

Paso 1: el HTML

Asumo que tienes algo así:

```
<input type="text" id="entrada" placeholder="Escribe un mensaje...">
<button id="enviar">Enviar</button>
```

Paso 2: validar antes de enviar

```
const entrada = document.getElementById("entrada");
const botonEnviar = document.getElementById("enviar");

function enviarMensaje() {
  const texto = entrada.value.trim();
  if (texto === "") return;

  mostrarMensaje(texto);
  entrada.value = "";
  entrada.focus();
}

botonEnviar.addEventListener("click", enviarMensaje);
```

Qué hace cada parte:

- `entrada.value` toma lo que el usuario escribió.
- `.trim()` borra los espacios al inicio y al final. Sin esto, alguien podría enviar un mensaje de solo espacios y tu validación lo dejaría pasar.
- `if (texto === "") return;` es la validación: si no hay texto, `return` sale de la función y nada de lo que sigue se ejecuta.
- Si el texto es válido, se muestra, se limpia el campo y el cursor vuelve al campo para seguir escribiendo.

Paso 3: enviar también con la tecla Enter

En un chat es normal enviar con Enter, y como usa la misma función, la validación también se aplica:

```
entrada.addEventListener("keydown", (e) => {
  if (e.key === "Enter") enviarMensaje();
});
```

Paso 4 (recomendado): desactivar el botón mientras está vacío

Esto le muestra al usuario visualmente que no puede enviar todavía:

```
botonEnviar.disabled = true;
```

```
entrada.addEventListener("input", () => {
  botonEnviar.disabled = entrada.value.trim() === "";
});
```

El evento input se dispara cada vez que el usuario escribe o borra. Si el campo queda vacío, el botón se desactiva, y si hay texto, se activa.

Y en el CSS, para que se note:

```
button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

Mantén la validación del paso 2 aunque uses el paso 4. El botón desactivado ayuda al usuario, pero la tecla Enter no depende del botón, así que la validación dentro de enviarMensaje es la que realmente te protege.

Cuando lo conectes a Gemini, esa llamada irá dentro de enviarMensaje, justo después de mostrarMensaje(texto), así que nunca se hará una petición con un mensaje vacío.

Kevin — mensaje 5

como pongo un indicador de "Escribiendo..." como tres puntos animados?

Claude — respuesta

El indicador es una burbuja con tres puntos que suben y bajan uno tras otro. La creas con JavaScript cuando el personaje empieza a "pensar" y la quitas cuando llega la respuesta.

Paso 1: el CSS de los puntos y la animación

```
.escribiendo {
  display: flex;
  gap: 4px;
  padding: 12px;
}

.escribiendo span {
  width: 8px;
  height: 8px;
  background: #888;
  border-radius: 50%;
  animation: rebote 1.2s infinite;
}

.escribiendo span:nth-child(2) { animation-delay: 0.2s; }
.escribiendo span:nth-child(3) { animation-delay: 0.4s; }

@keyframes rebote {
  0%, 60%, 100% { transform: translateY(0); opacity: 0.4; }
  30% { transform: translateY(-6px); opacity: 1; }
}
```

Qué hace cada parte:

- Cada span es un punto: un cuadrado de 8px con border-radius: 50%, que lo vuelve un círculo.
- @keyframes rebote define el movimiento: el punto sube 6px a la mitad de la animación y vuelve a bajar.
- animation-delay hace que el segundo y el tercer punto empiecen un poco después, y eso crea el efecto de "ola".

Paso 2: funciones para mostrar y quitar el indicador

```
function mostrarEscribiendo() {
  const indicador = document.createElement("div");
  indicador.className = "mensaje bot escribiendo";
  indicador.id = "indicador";
  indicador.innerHTML = "<span></span><span></span><span></span>";
  document.getElementById("mensajes").appendChild(indicador);
}
```

```

}

function quitarEscribiendo() {
  document.getElementById("indicador")?.remove();
}

```

- La burbuja lleva también la clase `mensaje bot`, así que se ve igual que los mensajes del personaje.
- El `id="indicador"` permite encontrarla después para borrarla.
- Aquí sí puedes usar `innerHTML` sin riesgo, porque el contenido lo escribes tú, no viene del usuario ni de la IA.
- El `?.` en `quitarEscribiendo` evita un error si el indicador ya no existe.

Paso 3: usarlo en tu función de enviar

Como todavía no tienes Gemini conectado, puedes simular la espera con una pausa de 2 segundos para probar que todo funciona:

```

async function enviarMensaje() {
  const texto = entrada.value.trim();
  if (texto === "") return;

  mostrarMensaje(texto);
  entrada.value = "";

  mostrarEscribiendo();
  await new Promise(r => setTimeout(r, 2000)); // simula la espera
  quitarEscribiendo();

  mostrarMensaje("Respuesta de prueba del personaje");
}

```

La palabra `async` antes de `function` es la que permite usar `await`, que hace que el código espere a que termine la pausa antes de seguir. Cuando conectes Gemini, reemplazarás la línea del `setTimeout` por la llamada real a la API y el indicador funcionará igual.

Paso 4 (recomendado): bajar el scroll automáticamente

Si el chat ya tiene muchos mensajes, el indicador puede quedar fuera de la vista. Agrega esta línea al final de `mostrarEscribiendo` y de `mostrarMensaje`:

```

const contenedor = document.getElementById("mensajes");
contenedor.scrollTop = contenedor.scrollHeight;

```

Esto funciona si tu contenedor `#mensajes` tiene una altura definida y `overflow-y: auto` en el CSS.

Un detalle más: mientras el indicador está visible, conviene desactivar el botón de enviar para que el usuario no mande otro mensaje antes de recibir la respuesta. Si quieres, en la siguiente pregunta te muestro cómo combinarlo con lo que ya hiciste.

Kevin — mensaje 6

con que función puedo pintar la vista de una página cuando uso los botones de atrás y adelante en el navegador?

Claude — respuesta

No existe una función del navegador que "pinte" la vista por sí sola. Lo que existe es un **evento** que te avisa cuando el usuario pulsa atrás o adelante, y en ese momento llamas a una función tuya que muestra la vista correcta. Hay dos formas de hacerlo:

- **hashchange**: usa rutas con `#`, como `index.html#/chat`. Es la más sencilla y funciona sin configurar nada, incluso abriendo el archivo directamente o en GitHub Pages.
- **popstate** con `history.pushState`: usa rutas limpias como `/chat`, pero al recargar la página el servidor debe estar configurado para devolver siempre `index.html`, o mostrará un error 404.

Para tu proyecto te recomiendo `hashchange`, porque te da el mismo resultado con menos problemas.

Paso 1: las vistas en el HTML

Cada "página" es una sección dentro del mismo index.html:

```
<nav>
  <a href="#/inicio">Inicio</a>
  <a href="#/chat">Chat</a>
  <a href="#/acerca">Acerca</a>
</nav>

<section id="vista-inicio" class="vista">...</section>
<section id="vista-chat" class="vista">...</section>
<section id="vista-acerca" class="vista">...</section>
```

Los enlaces con # no recargan la página. Solo cambian la URL y quedan guardados en el historial del navegador.

Paso 2: ocultar las vistas por defecto en el CSS

```
.vista { display: none; }
.vista.activa { display: block; }
```

Paso 3: la función que pinta la vista

```
function renderizar() {
  const ruta = location.hash.replace("#/", "") || "inicio";

  document.querySelectorAll(".vista").forEach(v => v.classList.remove("activa"));

  const vista = document.getElementById("vista-" + ruta);
  if (vista) {
    vista.classList.add("activa");
  } else {
    document.getElementById("vista-inicio").classList.add("activa");
  }
}
```

Qué hace cada parte:

- location.hash lee la parte de la URL que empieza con #. Si la URL es index.html#/chat, devuelve "#/chat", y con replace queda solo "chat".
- || "inicio" hace que, si no hay hash (por ejemplo, al abrir la página por primera vez), se muestre la vista de inicio.
- Luego se quita la clase activa de todas las vistas y se le pone solo a la que corresponde.
- Si alguien escribe una ruta que no existe, como #/abc, se muestra inicio en vez de una página en blanco.

Paso 4: conectar la función a los eventos

```
window.addEventListener("hashchange", renderizar);
window.addEventListener("DOMContentLoaded", renderizar);
```

- hashchange se dispara cada vez que cambia el # de la URL: cuando el usuario hace clic en un enlace, y también cuando pulsa **atrás o adelante** en el navegador. Esa es la respuesta a tu pregunta.
- DOMContentLoaded pinta la vista correcta al cargar la página. Así, si alguien recarga estando en #/chat, sigue viendo el chat.

Un detalle importante para tu chatbot

Como las vistas solo se ocultan y no se borran, los mensajes del chat no se pierden al ir a otra vista y volver. Eso es justo lo que quieres en una SPA.

Si más adelante tu proyecto te exige usar rutas sin #, te explico la versión con popstate y pushState.

Kevin — mensaje 7

y popstate?

Claude — respuesta

popstate hace lo mismo que hashchange, pero con rutas limpias como /chat en vez de #/chat. La diferencia principal es que aquí tú tienes que hacer más trabajo manual, porque el navegador no cambia la URL solo.

La idea general

- Cuando el usuario hace clic en un enlace, tú cambias la URL con `history.pushState` y pintas la vista.
- Cuando el usuario pulsa atrás o adelante, el navegador dispara `popstate` y tú vuelves a pintar la vista.

Algo clave: `pushState` cambia la URL pero **no** dispara `popstate`. Por eso, después de usarlo, tienes que llamar tú mismo a la función que pinta.

Paso 1: los enlaces en el HTML

```
<nav>
  <a href="/inicio" data-enlace>Inicio</a>
  <a href="/chat" data-enlace>Chat</a>
  <a href="/acerca" data-enlace>Acerca</a>
</nav>
```

El atributo `data-enlace` sirve para identificar qué enlaces maneja tu SPA. Las secciones y el CSS de las vistas son los mismos de la respuesta anterior.

Paso 2: la función que pinta la vista

```
function renderizar() {
  const ruta = location.pathname.replace("/", "") || "inicio";

  document.querySelectorAll(".vista").forEach(v => v.classList.remove("activa"));

  const vista = document.getElementById("vista-" + ruta);
  if (vista) {
    vista.classList.add("activa");
  } else {
    document.getElementById("vista-inicio").classList.add("activa");
  }
}
```

Es casi igual a la versión con hash. El único cambio es que ahora lees `location.pathname`, que es la parte de la URL después del dominio. Si la URL es `misitio.com/chat`, devuelve `/chat`.

Paso 3: interceptar los clics

Normalmente, un enlace hace que el navegador cargue otra página. Hay que impedirlo:

```
document.addEventListener("click", (e) => {
  const enlace = e.target.closest("[data-enlace]");
  if (!enlace) return;

  e.preventDefault();
  history.pushState(null, "", enlace.getAttribute("href"));
  renderizar();
});
```

- `closest("[data-enlace]")` busca si el clic fue en uno de tus enlaces (o en algo dentro de uno).
- `e.preventDefault()` cancela la recarga de página.
- `history.pushState(null, "", "/chat")` cambia la URL y guarda una entrada nueva en el historial, sin recargar.
- `renderizar()` pinta la vista, porque `pushState` no lo hace por ti.

Paso 4: escuchar atrás y adelante

```
window.addEventListener("popstate", renderizar);
window.addEventListener("DOMContentLoaded", renderizar);
```

popstate se dispara cuando el usuario pulsa atrás o adelante. El navegador ya cambió la URL, así que solo tienes que volver a pintar.

El problema que debes conocer

Si el usuario está en `misitio.com/chat` y recarga la página, el navegador le pide al servidor un archivo llamado `chat`, que no existe, y aparece un error 404. Para evitarlo, el servidor debe estar configurado para devolver siempre `index.html` en cualquier ruta. Cada hosting lo hace de forma distinta; por ejemplo, en Netlify se crea un archivo llamado `_redirects` con esta línea:

```
/* /index.html 200
```

Además, esta versión no funciona si abres el `index.html` haciendo doble clic, porque necesita un servidor. En VS Code puedes usar la extensión Live Server para probarlo.

¿Cuál usar?

Si tu proyecto no te exige URLs sin #, quédate con hashchange: hace lo mismo con menos configuración. Usa popstate si te piden URLs limpias o si el profesor lo especifica.

Kevin — mensaje 8

Y si en mi segundo argumento uso una función que me permite renderizar rutas?

Claude — respuesta

Sí, y de hecho eso es exactamente lo que estabas haciendo. En `addEventListener`, el segundo argumento es la función que se ejecuta cuando ocurre el evento. Puede ser cualquier función tuya, incluida una que se encargue de renderizar las rutas.

```
window.addEventListener("popstate", renderizarRuta);
```

El error más común

Pasa la función **sin paréntesis**:

```
window.addEventListener("popstate", renderizarRuta); // correcto
window.addEventListener("popstate", renderizarRuta()); // incorrecto
```

Sin paréntesis le estás entregando la función al navegador para que él la ejecute cuando el usuario pulse atrás o adelante. Con paréntesis, la función se ejecuta en ese mismo momento, una sola vez, y lo que recibe el evento es el resultado (normalmente `undefined`), así que atrás y adelante dejan de funcionar.

Una versión más ordenada: un objeto de rutas

Si tu función de renderizar va a crecer, una forma eficiente es guardar las rutas en un objeto, donde cada ruta apunta a la función que dibuja esa vista:

```
const rutas = {
  "/inicio": vistaInicio,
  "/chat": vistaChat,
  "/acerca": vistaAcerca
};

function renderizarRuta() {
  const vista = rutas[location.pathname] || vistaInicio;
  vista();
}
```

- `rutas[location.pathname]` busca en el objeto la función que corresponde a la URL actual. Si estás en `/chat`, obtiene `vistaChat`.
- `|| vistaInicio` cubre las rutas que no existen.
- `vista()` ejecuta la función encontrada.

Cada vista puede ser una función que muestre su sección, como en las respuestas anteriores:

```
function mostrarSeccion(id) {
  document.querySelectorAll(".vista").forEach(v => v.classList.remove("activa"));
  document.getElementById(id).classList.add("activa");
}

function vistaInicio() { mostrarSeccion("vista-inicio"); }
function vistaChat() { mostrarSeccion("vista-chat"); }
function vistaAcerca() { mostrarSeccion("vista-acerca"); }
```

La ventaja es que, para agregar una página nueva, solo creas su función y añades una línea al objeto rutas, sin tocar `renderizarRuta`.

Luego la conectas igual que antes, en los tres lugares:

```
window.addEventListener("popstate", renderizarRuta);
window.addEventListener("DOMContentLoaded", renderizarRuta);
// y dentro del manejador de clics, después de history.pushState(...)
renderizarRuta();
```

Si con "segundo argumento" te referías al de `history.pushState(estado, titulo, url)`, ese no acepta funciones: es un título que los navegadores ignoran, por eso se pone "". La función de `renderizar` siempre va en `addEventListener` o la llamas tú después de `pushState`.

Kevin — mensaje 9

Para que sirve exactamente `navigateTo()` ? solo cambia la url o hace algo más?

Claude — respuesta

`navigateTo()` no es una función del navegador. Es una función que **tú creas** (o que viene en el tutorial o código que estás siguiendo), así que hace exactamente lo que esté escrito dentro de ella. Es un nombre muy común en ejemplos de SPA con JavaScript puro, y casi siempre hace dos cosas, no solo una:

```
function navigateTo(url) {
  history.pushState(null, "", url); // 1. cambia la URL
  renderizarRuta();                // 2. pinta la vista
}
```

1. `history.pushState` cambia la URL y guarda la entrada en el historial, sin recargar la página.
2. `renderizarRuta()` muestra la vista que corresponde a esa nueva URL.

Si solo tuviera la primera línea, la URL cambiaría pero la pantalla seguiría igual, porque `pushState` no pinta nada por sí solo. Por eso se juntan las dos en una función.

Para qué sirve tenerla

Es una forma de no repetir esas dos líneas en cada lugar donde navegas. Por ejemplo, tu manejador de clics queda así:

```
document.addEventListener("click", (e) => {
  const enlace = e.target.closest("[data-enlace]");
  if (!enlace) return;

  e.preventDefault();
  navigateTo(enlace.getAttribute("href"));
});
```

Y también puedes navegar desde cualquier parte de tu código, no solo desde enlaces. Por ejemplo, un botón "Empezar a chatear" en la página de inicio:

```
botonEmpezar.addEventListener("click", () => navigateTo("/chat"));
```

Lo que NO hace

No se usa para atrás y adelante. En ese caso el navegador ya cambió la URL, así que solo hay que pintar, y por eso en `popstate` se pasa directamente `renderizarRuta`, no `navigateTo`:

```
window.addEventListener("popstate", renderizarRuta);
```

Si usaras `navigateTo` ahí, se agregaría una entrada nueva al historial cada vez que el usuario pulsa atrás, y el botón dejaría de funcionar bien.

En resumen: `navigateTo` es para cuando **tu código** decide cambiar de página (cambia la URL y pinta), y `renderizarRuta` sola es para cuando **el navegador** ya cambió la URL (solo pinta).

Kevin — mensaje 10

Como puedo enviar el historial completo al backend?

Claude — respuesta

Para enviar el historial completo necesitas guardarlo en un arreglo mientras el usuario chatea, y luego mandarlo al backend en cada petición. Esto es necesario porque Gemini no recuerda nada entre una petición y otra: si solo le envías el último mensaje, el personaje "olvida" toda la conversación.

Paso 1: crear el arreglo del historial

```
const historial = [];
```

Te recomiendo guardar cada mensaje directamente con el formato que usa Gemini, así el backend no tiene que transformarlo:

```
{ role: "user", parts: [{ text: "Hola, ¿quién eres?" }] }
{ role: "model", parts: [{ text: "Soy el guardián del bosque." }] }
```

`role` indica quién habló: `"user"` es el usuario y `"model"` es el personaje.

Paso 2: una función para agregar mensajes

```
function agregarAlHistorial(rol, texto) {
  historial.push({ role: rol, parts: [{ text: texto }] });
}
```

Paso 3: enviar el historial al backend

```
async function pedirRespuesta() {
  const respuesta = await fetch("/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ historial })
  });

  if (!respuesta.ok) throw new Error("Error del servidor");

  const data = await respuesta.json();
  return data.texto;
}
```

- `fetch` hace la petición al backend. Cambia `"/api/chat"` por la ruta de tu servidor.
- `method: "POST"` se usa porque estás enviando datos.
- `headers` le avisa al backend que los datos van en formato JSON.
- `JSON.stringify` convierte el arreglo en texto, porque por la red solo viaja texto.
- `respuesta.ok` revisa que el servidor no haya respondido con un error.
- Aquí asumo que tu backend devuelve `{ texto: "..."`. Si devuelve la respuesta completa de Gemini, usa la función `obtenerTexto(data)` que vimos antes.

Paso 4: conectarlo con tu función de enviar

```
async function enviarMensaje() {
  const texto = entrada.value.trim();
  if (texto === "") return;

  mostrarMensaje(texto);
  agregarAlHistorial("user", texto);
  entrada.value = "";
}
```

```

mostrarEscribiendo();
try {
  const textoBot = await pedirRespuesta();
  agregarAlHistorial("model", textoBot);
  mostrarMensaje(textoBot);
} catch (error) {
  historial.pop();
  mostrarMensaje("No se pudo obtener respuesta. Intenta de nuevo.");
} finally {
  quitarEscribiendo();
}
}

```

- El mensaje del usuario se agrega al historial **antes** de pedir la respuesta, para que vaya incluido en el envío.
- La respuesta del personaje se agrega solo si llegó bien.
- Si hay un error, `historial.pop()` quita el último mensaje del usuario. Gemini espera que los turnos se alternen entre usuario y modelo, y dos mensajes seguidos del usuario podrían causar problemas en la siguiente petición.
- `finally` se ejecuta siempre, haya error o no, así que el indicador de "Escribiendo..." se quita en ambos casos.

Paso 5 (recomendado): limitar el tamaño del historial

En conversaciones largas, enviar todo cada vez hace las peticiones más lentas y gasta más cuota de la API. Puedes enviar solo los últimos mensajes:

```
body: JSON.stringify({ historial: historial.slice(-20) })
```

`slice(-20)` toma los últimos 20 mensajes, que son 10 intercambios. Usa un número par para que el recorte no rompa la alternancia entre usuario y modelo.

Del lado del backend

El backend recibe `historial` y se lo pasa a Gemini en el campo `contents` de la petición. Ahí mismo es donde conviene poner la instrucción del personaje (`systemInstruction`), en vez de mandarla desde el navegador, porque así nadie puede modificar la personalidad desde las herramientas de desarrollador. Si me dices con qué estás haciendo el backend, te muestro esa parte.

Kevin — mensaje 11

Que formato usa gemini para enviar sus mensajes? sus mensajes

Claude — respuesta

Gemini responde con un objeto JSON. El mensaje del personaje viene en el mismo formato que usas para el historial, pero envuelto dentro de otros datos. Una respuesta típica se ve así (simplificada):

```

{
  "candidates": [
    {
      "content": {
        "role": "model",
        "parts": [
          { "text": "Soy el guardián del bosque. ¿Qué buscas, viajero?" }
        ]
      },
      "finishReason": "STOP"
    }
  ],
  "usageMetadata": {
    "promptTokenCount": 42,
    "candidatesTokenCount": 15,
    "totalTokenCount": 57
  }
}

```

Qué significa cada parte

candidates es un arreglo de posibles respuestas. Normalmente solo viene una, por eso se usa [0].

content es el mensaje en sí. Tiene role: "model", que indica que lo escribió la IA, y parts, un arreglo con el texto. Casi siempre hay una sola parte, pero puede llegar dividido en varias, por eso conviene unirlos.

finishReason explica por qué terminó la respuesta. Los valores más útiles son "STOP", que significa que terminó normalmente; "MAX_TOKENS", que significa que se cortó por llegar al límite de longitud; y "SAFETY", que significa que fue bloqueada por los filtros de seguridad.

usageMetadata dice cuántos tokens se usaron. Te sirve para medir el consumo de tu cuota, pero no lo necesitas para el chat.

El texto viene en Markdown

Dentro de text, Gemini suele usar Markdown: ****negrita****, **cursiva**, listas con guiones, etc. Por eso antes vimos cómo pedirle texto plano o limpiar esos símbolos.

Un atajo para tu historial

Como content ya tiene exactamente el formato { role, parts } que guardas en tu historial, puedes agregarlo directamente sin reconstruirlo:

```
const mensaje = data.candidates[0].content;
historial.push(mensaje);
```

Cuando la respuesta viene bloqueada

Si Gemini bloquea tu pregunta, puede que candidates no venga, o que venga sin parts. En ese caso aparece un campo promptFeedback con el motivo. Por eso la función obtenerTexto que vimos usa ?. para que tu app muestre un mensaje de error en vez de romperse. También puedes revisar finishReason para avisarle al usuario:

```
const candidato = data?.candidates?.[0];

if (!candidato || candidato.finishReason === "SAFETY") {
  return "El personaje prefiere no responder a eso.";
}
```

Así el mensaje de error encaja con la personalidad del personaje en vez de parecer una falla técnica.

Kevin — mensaje 12

Como conecto mi rol de character al rol de model de la IA?

Claude — respuesta

Gemini solo acepta dos roles: "user" y "model". Si en tu aplicación usas "character" para los mensajes del personaje, Gemini no lo va a entender y la petición fallará. La solución es **traducir** los roles justo antes de enviar el historial, sin cambiar cómo los guardas en tu app.

Paso 1: tu historial con tus propios roles

Supongamos que guardas los mensajes así:

```
const historial = [
  { role: "user", text: "Hola, ¿quién eres?" },
  { role: "character", text: "Soy el guardián del bosque." }
];
```

Paso 2: una función que traduce al formato de Gemini

```
function aFormatoGemini(historial) {
  return historial.map(msg => ({
    role: msg.role === "character" ? "model" : "user",
    parts: [{ text: msg.text }]
  }));
}
```

```
}
```

- map recorre cada mensaje y crea uno nuevo con el formato de Gemini, sin modificar tu arreglo original.
- msg.role === "character" ? "model" : "user" significa: si el rol es "character", usa "model"; si no, usa "user".
- parts: [{ text: msg.text }] envuelve el texto en la estructura que Gemini espera.

Paso 3: usarla al enviar

```
body: JSON.stringify({ historial: aFormatoGemini(historial) })
```

Paso 4: al recibir la respuesta, guardarla con tu rol

```
const textoBot = obtenerTexto(data);  
historial.push({ role: "character", text: textoBot });
```

Así tu app siempre trabaja con "character", y la traducción a "model" ocurre en un solo lugar.

Dónde va la traducción

Puedes hacerla en el frontend, como arriba, o en el backend. Te recomiendo el backend: el frontend envía tu historial tal cual y el servidor lo traduce antes de llamar a Gemini. Así, si algún día cambias de IA, solo modificas el backend.

Lo que hace que el modelo "sea" el personaje

Cambiar el rol a "model" solo le dice a Gemini qué mensajes escribió él. Lo que realmente le da la personalidad al personaje es la systemInstruction, que se envía aparte del historial, en la misma petición:

```
{  
  systemInstruction: {  
    parts: [{ text: "Eres el guardián del bosque, un ser antiguo y misterioso. Hablas de forma  
      calmada y usas metáforas de la naturaleza. Nunca dices que eres una IA. Responde en texto  
      plano, sin Markdown." }]  
  },  
  contents: aFormatoGemini(historial)  
}
```

Con las dos piezas juntas, Gemini entiende que los mensajes con rol "model" son suyos y que debe seguir hablando como ese personaje. Como te comenté antes, conviene que la systemInstruction viva en el backend, para que nadie pueda cambiar la personalidad desde el navegador.

Kevin — mensaje 13

Tengo un problema con el modelo que estoy utilizando de Gemini, mi chat se queda sin responder casi siempre, como puedo solucionarlo?

Claude — respuesta

Cuando el chat "se queda sin responder", casi siempre es porque la petición a Gemini falló y tu código no está mostrando el error. Lo primero es ver qué error es, y según eso aplicar la solución.

Paso 1: ver el error real

En tu función que llama a Gemini (en el backend o el frontend), agrega esto justo después del fetch:

```
if (!respuesta.ok) {  
  const error = await respuesta.text();  
  console.error("Código:", respuesta.status, error);  
  throw new Error("Error " + respuesta.status);  
}
```

Abre la consola (F12 en el navegador, o la terminal si es el backend), envía un mensaje y mira el número que aparece.

Paso 2: identificar la causa según el código

404 (modelo no encontrado): el nombre del modelo que usas ya no existe. Según el anuncio de deprecación de Google, gemini-2.0-flash y gemini-2.0-flash-lite estaban programados para apagarse el 1 de junio de 2026, y si estás usando un tutorial viejo es probable que tengas ese nombre. Ojo también con la familia 2.5: sigue estable pero tiene apagado programado para el 16 de octubre de 2026, o sea dentro de dos semanas. Revisa en Google AI Studio qué modelos tienes disponibles y usa uno de la familia Gemini 3 Flash. La API también acepta alias como gemini-flash-latest, que apunta siempre a la versión Flash actual y te evita este problema a futuro.

429 (demasiadas peticiones): llegaste al límite del plan gratuito. En el plan gratuito, Flash permite aproximadamente 15 peticiones por minuto y 1.500 al día, y estos números pueden cambiar. Además, los modelos Pro salieron del plan gratuito en abril de 2026, así que si usas uno Pro sin facturación activada, fallará. Tus límites exactos los puedes ver en tu cuenta de AI Studio.

503 (servidor sobrecargado): Google está saturado en ese momento. No es culpa de tu código; se soluciona reintentando.

200 pero sin texto: la petición funcionó pero la respuesta llegó vacía. Revisa el finishReason. Si dice "SAFETY", se bloqueó por los filtros. Si dice "MAX_TOKENS" y no hay texto, es porque los modelos recientes "piensan" antes de responder y ese pensamiento consume parte del límite; si pusiste un maxOutputTokens bajo, súbelo o quítalo.

Paso 3: hacer tu código resistente a fallos

Esta función reintentará automáticamente cuando el error es temporal (429 o 503) y corta la espera si Gemini tarda demasiado:

```
async function llamarGemini(cuerpo, intentos = 3) {
  for (let i = 0; i < intentos; i++) {
    const respuesta = await fetch(URL_GEMINI, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(cuerpo),
      signal: AbortSignal.timeout(30000)
    });

    if (respuesta.ok) return await respuesta.json();

    if (respuesta.status !== 429 && respuesta.status !== 503) {
      throw new Error("Error " + respuesta.status);
    }

    await new Promise(r => setTimeout(r, 2000 * (i + 1)));
  }
  throw new Error("Gemini no respondió después de varios intentos");
}
```

- `AbortSignal.timeout(30000)` cancela la petición si pasan 30 segundos sin respuesta, así tu chat no se queda esperando para siempre.
- Si la respuesta es correcta, se devuelve de inmediato.
- Si el error no es temporal (por ejemplo, un 404), no tiene sentido reintentar, así que lanza el error directamente.
- Si es 429 o 503, espera y vuelve a intentar. La espera crece en cada intento (2, 4 y 6 segundos), lo que le da tiempo a Google para liberarse.

Paso 4: que el usuario siempre vea algo

Asegúrate de que tu `enviarMensaje` tenga el `try / catch / finally` que vimos antes. Así, aunque todo falle, el indicador de "Escribiendo..." desaparece y el usuario ve un mensaje de error en lugar de un chat congelado.

Si me dices qué código de error te aparece en la consola y qué nombre de modelo estás usando, te digo exactamente qué cambiar.

Kevin — mensaje 14

Es el error 503, no puedo usar varios modelos de gemini para evitarlo?

Claude — respuesta

Sí, y es una buena solución. Se llama **modelo de respaldo** (fallback): si el primer modelo responde 503, tu código prueba automáticamente con otro. Funciona bien porque la saturación suele afectar a un modelo específico, no a todos a la vez. Además, en el plan gratuito cada modelo tiene su propio límite, así que también te ayuda con los errores 429.

Paso 1: la lista de modelos en orden de preferencia

```
const MODELOS = [
  "gemini-3-flash-preview",
  "gemini-3.1-flash-lite",
  "gemini-flash-latest"
];
```

El primero es el que prefieres; los demás son respaldos. Estos nombres son ejemplos: en la familia Gemini 3, gemini-3.1-flash-lite es estable desde mayo de 2026, mientras que gemini-3-flash-preview sigue en vista previa. Confirma en Google AI Studio cuáles tienes disponibles en tu cuenta, porque los nombres cambian con frecuencia. Te recomiendo no incluir modelos 2.5, porque se apagan este mes.

Paso 2: la función que prueba cada modelo

```
async function llamarGemini(cuerpo) {
  for (const modelo of MODELOS) {
    const url =
      `https://generativelanguage.googleapis.com/v1beta/models/${modelo}:generateContent`;

    try {
      const respuesta = await fetch(url, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          "x-goog-api-key": API_KEY
        },
        body: JSON.stringify(cuerpo),
        signal: AbortSignal.timeout(30000)
      });

      if (respuesta.ok) return await respuesta.json();

      if (respuesta.status !== 503 && respuesta.status !== 429) {
        throw new Error("Error " + respuesta.status);
      }

      console.warn(`${modelo} no disponible (${respuesta.status}), probando el siguiente...`);
    } catch (error) {
      if (error.name !== "TimeoutError") throw error;
      console.warn(`${modelo} tardó demasiado, probando el siguiente...`);
    }
  }

  throw new Error("Ningún modelo respondió");
}
```

Qué hace cada parte:

- for (const modelo of MODELOS) recorre la lista en orden.
- La URL se arma con el nombre del modelo, así que cambiar de modelo es solo cambiar ese texto. El resto de la petición (historial, systemInstruction) es igual para todos.
- Si la respuesta es correcta, se devuelve y el ciclo termina ahí; los demás modelos no se usan.
- Si el error es 503 o 429, muestra un aviso en la consola y pasa al siguiente modelo.
- Si el error es otro, como un 400 por un problema en tu petición, lanza el error de inmediato, porque cambiar de modelo no lo arreglaría.

- El catch también pasa al siguiente modelo si uno tarda más de 30 segundos.
- Si todos fallan, lanza un error final que tu enviarMensaje atrapa para mostrarle un mensaje al usuario.

Paso 3: usarla

No tienes que cambiar nada más. Donde antes llamabas a Gemini, ahora llamas a esta función:

```
const data = await llamarGemini({
  systemInstruction: { parts: [{ text: INSTRUCCION_PERSONAJE }] },
  contents: aFormatoGemini(historial)
});
```

Dos cosas a tener en cuenta

Los modelos Lite son más rápidos y tienen límites más altos, pero sus respuestas pueden ser un poco más simples. Como la personalidad viene de la systemInstruction, el personaje seguirá siendo el mismo; solo puede notarse algo de diferencia en la calidad.

Este código usa la API key, así que debe ir en el backend, no en el JavaScript del navegador, por lo que te comenté al principio sobre que la clave quedaría visible.

Kevin — mensaje 15

Como tengo que modificar mi Handler para agregar estos modelos de repuesto

Claude — respuesta

No puedo ver tu handler, así que te muestro cómo queda uno típico (el formato que usan Vercel y otros servicios serverless con Node.js) y qué partes cambian. Si el tuyo es diferente, pégalo y te indico exactamente dónde va cada cosa.

Lo que probablemente tienes ahora

Un handler que hace un solo fetch a un modelo fijo:

```
const url = "https://generativelanguage.googleapis.com/v1beta/models/gemini-x:generateContent";
const respuesta = await fetch(url, { ... });
```

El cambio consiste en reemplazar ese fetch por la función llamarGemini, que es la que prueba los modelos en orden.

Cómo queda el handler completo

```
const API_KEY = process.env.GEMINI_API_KEY;

const MODELOS = [
  "gemini-3-flash-preview",
  "gemini-3.1-flash-lite",
  "gemini-flash-latest"
];

const INSTRUCCION_PERSONAJE = "Eres el guardián del bosque... Responde en texto plano, sin Markdown.";

async function llamarGemini(cuerpo) {
  // la misma función de la respuesta anterior
}

function aFormatoGemini(historial) {
  return historial.map(msg => ({
    role: msg.role === "character" ? "model" : "user",
    parts: [{ text: msg.text }]
  }));
}

export default async function handler(req, res) {
  if (req.method !== "POST") {
    return res.status(405).json({ error: "Método no permitido" });
  }
}
```

```

const { historial } = req.body;
if (!Array.isArray(historial) || historial.length === 0) {
  return res.status(400).json({ error: "Historial vacío" });
}

try {
  const data = await llamarGemini({
    systemInstruction: { parts: [{ text: INSTRUCCION_PERSONAJE }] },
    contents: aFormatoGemini(historial.slice(-20))
  });

  const partes = data?.candidates?.[0]?.content?.parts;
  const texto = partes ? partes.map(p => p.text || "").join("") : "";

  if (!texto) {
    return res.status(200).json({ texto: "El personaje prefiere no responder a eso." });
  }

  res.status(200).json({ texto });
} catch (error) {
  console.error(error);
  res.status(503).json({ error: "El personaje no está disponible ahora. Intenta en un momento." });
}
}

```

Qué hace cada parte

- `process.env.GEMINI_API_KEY` lee la clave desde las variables de entorno del servidor, así nunca aparece escrita en tu código. La configuras en el panel de tu hosting o en un archivo `.env` para pruebas locales.
- `MODELOS`, `INSTRUCCION_PERSONAJE`, `llamarGemini` y `aFormatoGemini` van **fuera** del handler. Así se definen una sola vez y el handler queda corto y fácil de leer.
- La primera validación rechaza cualquier petición que no sea POST.
- La segunda revisa que el historial exista y tenga mensajes, para no gastar una llamada a Gemini en vano.
- Dentro del `try`, `llamarGemini` se encarga de probar los modelos. El handler no necesita saber cuál respondió.
- Luego se extrae el texto y se devuelve como `{ texto }`, que es justo lo que espera tu `pedirRespuesta()` del frontend.
- Si todos los modelos fallan, el `catch` responde con un error 503 y un mensaje claro. Tu frontend lo recibe, entra en su propio `catch` y muestra el mensaje al usuario.

Lo que no cambia

Tu frontend sigue igual: envía el historial a la misma ruta y recibe `{ texto }`. Toda la lógica de los modelos de repuesto queda escondida en el backend, que es justamente donde debe estar.

Si tu handler usa otro formato (por ejemplo Express con `app.post`, o Netlify Functions con `exports.handler`), pégamelo y te lo adapto.

Kevin — mensaje 16

Me puedes decir los modelos de gemini más rápidos para incluirlos en mi constante de modelos?

Claude — respuesta

Los más rápidos de Gemini son los de la línea **Flash-Lite**, y le siguen los **Flash**. Estos son los que te recomiendo hoy:

gemini-3.5-flash-lite: es el más reciente de la línea Flash-Lite, optimizado para inferencia rápida a costo mínimo. Es estable (GA) desde el 21 de julio de 2026 y su retiro está previsto para julio de 2027 o después, así que no tendrás el problema de nombres que dejan de funcionar.

gemini-3.1-flash-lite: Google lo describe como su modelo más eficiente en costo, optimizado para baja latencia y alto volumen de tráfico. Es estable desde mayo de 2026, con retiro no antes de mayo de 2027. Es un buen primer respaldo.

gemini-3.8-flash: es un poco más lento que los Lite, pero más inteligente. Google indica que para proyectos nuevos se usen sus modelos más recientes: 3.5 Flash-Lite o 3.8 Flash. Sirve como último respaldo, porque es de otra línea y es menos probable que esté saturado al mismo tiempo que los Lite.

Tu constante quedaría así:

```
const MODELOS = [  
  "gemini-3.5-flash-lite",  
  "gemini-3.1-flash-lite",  
  "gemini-3.8-flash"  
];
```

El orden importa: tu función prueba primero el más rápido y solo pasa a los siguientes si hay error 503 o 429.

Corrección a mi respuesta anterior

Te había sugerido `gemini-3-flash-preview` como primera opción. Ya no es lo ideal: los modelos con `preview` en el nombre pueden desaparecer sin mucho aviso (por ejemplo, la versión `preview` de Gemini 3.1 Flash-Lite ya fue apagada). Para tu proyecto usa siempre modelos estables, sin `preview` en el nombre.

Antes de usarlos

Entra a Google AI Studio y confirma que los tres aparecen disponibles para tu API key en el plan gratuito, porque la disponibilidad puede variar por cuenta. Si alguno no aparece, simplemente quítalo de la lista. Evita los modelos Pro: son más lentos y no están en el plan gratuito.

Kevin — mensaje 17

Necesito exportar este archivo como evidencia de uso de IA, puedes por favor crear un pdf de este chat? no recortes respuestas